

Architecture and trade-offs

This document is about the ledger in this repository: what the build does today, where that stops working, and what I left out to stay in scope.

1. Append-only at scale

What breaks first. The evening fee check. Every evening, for every account it can charge, it goes back to Day 1 and tests each day in turn. For a day with no fee yet, which is nearly every day, that means reading the whole ledger twice: once to see whether the day already carries a fee, and once to add up its balance. The ledger is a single list shared by all accounts, so an account with ten entries still pays for everyone else's.

A hundred times the brief's events would not break this. A thousand events still replay in an instant. What breaks it is that the loop multiplies accounts by days by the size of the ledger, and all three grow together. With 1,000 accounts, a year of history and ten entries per account per day, the ledger holds 3.65 million entries, and by the end of the year each evening's check reads all of them 730,000 times, around 2.7 trillion entry visits. That is hours of work every evening, and more each day. The same habit shows elsewhere: finding active holds, finding the event a reversal names, and the daily report, which replays the stream once per day, all re-read everything.

First move: split by account and work in parallel. This is cheap because of something the model happens to guarantee: no event touches two accounts. There are no transfers, and balances, holds, fees and interest are all worked out one account at a time. So accounts can be dealt into groups, each with its own worker, and each account can keep its own entry list and outcome log. The only thing the groups share is the clock: every group finishes a day's events before any group closes that day. My ledger's rule that posting days never go backwards would then hold within a group, and the shared clock keeps its purpose, that a closed day takes no new postings. Each read shrinks from 3.65 million entries to the 3,650 that belong to one account, and the evening's work drops a thousandfold, to about 2.7 billion visits, spread across the workers.

What splitting leaves behind. It divides the work. It does not stop it growing. Three things in my design grow without limit.

- The entry list. This one is deliberate. An append-only ledger keeps everything, and a bank must retain it for years anyway.
- The outcome log, which keeps every declined and settled authorisation in the pile the next one searches.
- The set of open days. This is the one that hurts. No day is ever closed, so every evening re-tests every day since Day 1. Doubling an account's age quadruples its cost, however many workers there are.

The cheapest structural change: a balance brought forward. A bank statement starts from the balance brought forward, not from the account's first day. My code already works this way in miniature: every balance starts from the account's opening balance and adds entries to it. A date on that opening balance is the hook. Sums start from the brought-forward figure, the evening check starts from its date, and older entries move to cheaper storage without being lost. With a figure brought forward 30 days ago, the same evening check costs about 18 million visits, and it stays there however old the account gets.

I would introduce it in two steps, because they cost different things. First as a convenience: a late entry that reaches back past the brought-forward date discards the figure, which is rebuilt from the full log. No rule changes and no answer changes. Then as a closed period: once the bank agrees that nothing may be dated further back than, say, 30 days, the rare case disappears too. The price is that a genuinely old correction can no longer carry its true date and must be posted today as an explicit adjustment. That is a business decision, not a technical one, and section 2 returns to it.

2. Value-dated entries in production

A value date lets an entry count from an earlier day than the one it arrived on. The price of that is a closed day that is not closed. When a late entry arrives, my daily report prints a "restated" line listing the earlier days whose closing balance has just changed.

Operational surface.

- What the customer has seen, and what the bank has reported, become wrong. A statement already issued, or a figure already sent to finance or the regulator, described a day that has since changed.
- The ledger back-dates its own entries. I date a fee to the day it is for, so one late debit makes the ledger post further entries into the past. Dating a fee on the evening it is charged would avoid that, but it would change the figures in my replay, so I raise it as a question for the product owner and not as a fix.
- The build is lopsided. A late debit charges fees automatically. A late correction refunds nothing. KnownGapTest shows 75.00 of fees surviving the reversal of the debit that caused them. Section 4 returns to this.

Regulatory surface.

- Statements must be right. The Central Bank's Consumer Protection Standards require statements, and the calculations behind them, to be accurate. A late entry can make an accurate statement inaccurate after it was issued.
- The bank's own errors. Article 5 of the same Standards says that when the bank's error costs a customer money, the bank must refund it immediately, tell the customer within 10 complete business days, and must not benefit. If the late debit in my replay was the bank's mistake, my build does none of these.
- VAT on the fee, which I left out. An overdraft fee is an explicit charge, and explicit bank charges carry 5% VAT, where interest does not. My ledger charges a flat 25.00. In production the customer pays 1.25 on top, or the 25.00 includes it, and the brief does not say which. A fee dated into the past also raises the question of which tax period it belongs to.

The one control before going live: a statement never changes after the fact.

A statement shows each day as the bank knew it when the statement was issued, and it is never reissued. Anything that arrives later and counts from an earlier day appears on the next statement as an adjustment, showing the day it was posted, the day it counts from, and any fee, VAT or interest that moved because of it. That settles the tax question simply: a fee and its VAT belong to the period in which they were posted. It also fits the closed period in section 1: an old correction posted today and shown as an adjustment is exactly how this control treats every late entry. My ledger already supports this, because every entry keeps both the day it was posted and the day it counts from, and my daily report already prints each day once and shows later changes on the day they arrived.

3. Authorisation lifecycle

In my model an authorisation ends in only two ways: it is declined when it arrives, or it is settled. There is a third possibility that is not an ending at all. An authorisation that is approved and never settled keeps its hold forever, because nothing in my build expires or releases one.

That matters because the real world has endings my model cannot tell apart. A hotel or a car rental firm places a hold as a deposit. It may claim it, release it at checkout, or simply forget it, in which case the customer's bank lets it lapse, after about a week for most purchases and up to about a month for hotels and car rental. In my model the last two look the same: a hold that never goes away.

One assumption I made on purpose. A settlement for less than the hold releases the whole hold, and the unused part needs no separate reversal. That matches the common case, a hotel bill below the deposit or a fuel pump that holds a fixed amount and charges what was pumped, and is wrong for an order claimed in parts, the fifth row below.

What happens	Real-world case	My model today	What I would mandate
Declined	Not enough money	DECLINED outcome, no hold	Keep, and record the reason
Settled for more	A tip added after approval. A hotel minibar	Debited in full with no check	A tolerance by merchant type, with a top-up authorisation above it. The excess is paid but flagged
Released by the merchant	Checkout paid another way. A car returned undamaged. A cancelled order	Not possible. The hold stays	A RELEASED outcome that ends the hold, in full or in part
Expired	The merchant neither claims nor releases	No expiry. The money is reserved forever	An EXPIRED outcome appended at day close, after a period set by merchant type
Settled in parts	An order shipped in two parcels. A multi-leg ticket	The first settlement releases everything, so the second is rejected	A settlement marked as partial keeps the rest on hold until a final one, or expiry
Settled with no live authorisation	A purchase made offline in flight. A claim arriving after expiry	REJECTED and no money moves, as the brief requires	An exceptions queue, not a rejection. The bank is generally expected to honour it and dispute it afterwards

Every new ending is an appended outcome, like the ones I already have. The log stays append-only, and holds are still read from it.

4. What I cut and why

Some cuts are already covered above: reading the whole ledger for every answer and days that never close (section 1), the missing VAT and statements that can be restated (section 2), and holds that never expire, cannot be released and cannot be claimed in parts (section 3). The rest are below.

What I cut	Why it stayed out	The risk it defers
A refund of a fee when its cause is reversed	It depends on why the debit was reversed, and the event does not say	A customer stays out of pocket after the bank's own error, against Article 5. Until it is built: a daily list of fees on days that no longer close negative, each refunded or confirmed within 10 business days
A reason on a reversal	The brief's event carries none	A bank error and a merchant refund look identical, so nothing that depends on the difference can be automated
A limit on fees	The rule as given has none	An account 1.00 overdrawn is charged 25.00 every day, and each fee deepens the overdraft
A check for duplicates	A recorded stream arrives once	An event delivered twice is posted twice. Two holds with the same id are both released by one settlement
Checking the whole stream before applying any of it	The checks sit where the data is used	A malformed event stops a replay partway, leaving a day half applied. An unknown account posts but is never charged, never earns and never prints
Reversing anything but a plain credit or debit	It is the only case in the brief	No refund or chargeback of a settlement, and no reversal of an instalment credit
A calendar	The brief's window is Days 1 to 6	No cut-off time, time zone, weekend or holiday, all of which decide a real value date
The other side of each entry	The brief asks for an account ledger	Fees and interest come from nowhere and go nowhere, so nothing reconciles against the bank's own books
Storage and more than one thread	The brief asks for an in-memory build	No recovery except a full replay. Two events on one account at the same moment would race the available balance check